



Hybrid Intrusion Detection System for MQTT-Based IoT Networks Using Machine Learning

Graduation Project Report

Submitted in partial fulfillment of the requirements for the degree of
Bachelor in Artificial Intelligence Engineering – Information Security Systems

Prepared by:

Salam abedalkader abedalkader

Supervised by:

Dr. Waseem Juneidi

Eng. Mohammad Yamen Hallak

Academic Year:
2025 – 2026

Abstract

The rapid expansion of Internet of Things (IoT) systems has introduced significant security challenges, particularly due to the widespread use of lightweight communication protocols such as MQTT. These environments are highly dynamic and resource-constrained, making traditional intrusion detection systems ineffective in handling evolving and unknown attack patterns.

This work proposes a hybrid intrusion detection system for MQTT-based IoT networks that integrates multiple detection techniques within a unified framework. The system combines Random Forest for supervised classification, Isolation Forest for anomaly detection, and Kullback–Leibler (KL) Divergence for statistical analysis of traffic behavior. These components are fused using a stacking-based meta-model to produce a robust and adaptive decision mechanism.

To enhance adaptability, a dynamic thresholding method is introduced based on percentile analysis and exponential moving average (EMA) smoothing. This allows the system to continuously adjust its detection boundary according to changes in traffic distribution.

The system is implemented in a near real-time environment using a dataset-based streaming simulation, where network traffic is processed in chunks to approximate real-world conditions.

Experimental results demonstrate strong performance, achieving an accuracy of 99.36%, precision of 90.20%, recall of 98.14%, and an F1-score of 94.00%, while maintaining a low false positive rate of 0.57%. These results highlight the effectiveness of the proposed hybrid approach in detecting both known and unknown attacks.

Overall, the proposed system provides a robust, adaptive, and efficient solution for intrusion detection in MQTT-based IoT environments, demonstrating strong potential for practical deployment and future extension to real-time systems.

Keywords:

IoT Security, MQTT, Intrusion Detection System, Machine Learning, Hybrid Model, Anomaly Detection, Dynamic Threshold

Table of Contents

Abstract	1
Table of Contents	2
List of Figures	3
List of Tables	3
Glossary of Terms	4
Introduction	5
Problem Statement	6
Objectives	6
Chapter 1 — Theoretical Background	7
1.1 Internet of Things (IoT)	8
1.2 MQTT Protocol	8
1.3 Intrusion Detection Systems (IDS)	9
1.4 Machine Learning in Intrusion Detection	11
Chapter 2 — Related Work	15
Chapter 3 — Proposed System	18
3.1 System Architecture	19
3.2 Data Processing Pipeline	21
3.3 Feature Engineering	21
3.4 Hybrid Detection Model	25
3.5 Dynamic Threshold	27
3.6 Mathematical Model	29
3.7 Real-Time Implementation	32
Chapter 4 — Implementation and Experimental Setup	34
4.1 Dataset Description	35
4.2 Data Preparation	37
4.3 Model Training	39
4.4 Evaluation Setup	41
4.5 System Implementation (Streamlit)	42
Chapter 5 — Results and Discussion	45
5.1 Introduction	46
5.2 Quantitative Results	46
5.3 Real-Time Results Analysis	50
5.4 Discussion (Strengths and Limitations)	52
Chapter 6 — Conclusion and Future Work	54

6.1	Conclusion	55
6.2	Future Work.....	56
	References.....	57

List of Figures

Figure 1 :	MQTT publish/subscribe communication model.	9
Figure 2 :	Classification of intrusion detection systems.	10
Figure 3	Supervised and Unsupervised Machine Learning Workflows for Intrusion Detection Systems.	12
Figure 4 :	Proposed hybrid intrusion detection system architecture for MQTT-based IoT networks.	20
Figure 5:	Dynamic threshold adaptation over time based on anomaly score distribution.....	29
Figure 6:	Real-time intrusion detection dashboard implemented using Streamlit.....	43
Figure 7:	Confusion Matrix (Percentage of Total Dataset)	50

List of Tables

Table 1 :	Comparison of Related Work in IoT Intrusion Detection Systems	16
Table 2	Mapping of Selected Features to Supporting Research	22
Table 3:	Dataset Distribution of MQTTset	36
Table 4 :	Implementation Environment and Tools	37
Table 5 :	Feature Reduction and Selection Process.....	38
Table 6	Comparison of the Proposed Hybrid Model with Baseline Approaches	47
Table 7 :	Performance Evaluation Results on Test Dataset	48
Table 8 :	Real-Time Performance Across Chunks	52

Glossary of Terms

Term	Abbreviation	Description
Internet of Things	IoT	A network of interconnected devices that communicate and exchange data without human intervention
Message Queuing Telemetry Transport	MQTT	A lightweight publish/subscribe messaging protocol used in IoT environments
Intrusion Detection System	IDS	A system designed to monitor network traffic and detect malicious activities
Machine Learning	ML	A set of algorithms that learn patterns from data to make predictions or decisions
Random Forest	RF	A supervised learning algorithm based on an ensemble of decision trees for classification
Isolation Forest	IF	An unsupervised anomaly detection algorithm that isolates abnormal data points
Kullback–Leibler Divergence	KL Divergence	A statistical measure used to quantify the difference between probability distributions
Stacking Model	—	A meta-learning technique that combines multiple models using a secondary model
Dynamic Threshold	—	An adaptive threshold that changes based on data distribution
False Positive Rate	FPR	The proportion of normal samples incorrectly classified as attacks
True Positive Rate	TPR	The proportion of attacks correctly detected by the system
F1-score	—	A metric that balances precision and recall in classification tasks
Receiver Operating Characteristic	ROC	A curve representing the trade-off between true positive rate and false positive rate
Area Under Curve	AUC	A performance metric representing the area under the ROC curve
Precision	—	The proportion of correctly predicted attack samples among all predicted attacks
Recall	—	The proportion of actual attack samples that are correctly detected
Chunk Processing	—	A method of processing data in smaller segments for efficiency and scalability
Feature Engineering	—	The process of creating new features from raw data to improve model performance
Feature Selection	—	The process of selecting the most relevant features for model training

Introduction

The rapid expansion of Internet of Things (IoT) technologies has led to the deployment of a large number of interconnected devices across various domains, including smart homes, healthcare, and industrial systems. However, this growth has introduced significant security challenges due to the limited computational resources and weak built-in security mechanisms of IoT devices [1].

Among the commonly used communication protocols in IoT environments, the Message Queuing Telemetry Transport (MQTT) protocol has gained widespread adoption due to its lightweight nature and efficiency. Despite its advantages, MQTT lacks strong built-in security features, making it vulnerable to various types of cyber-attacks such as distributed denial-of-service (DDoS), flooding attacks, and abnormal traffic manipulation [2].

Traditional intrusion detection systems (IDS) are often not well-suited for IoT environments, as they rely on static rules or signatures and fail to adapt to the dynamic and evolving nature of network traffic. In addition, these systems may suffer from high false positive rates, which reduces their reliability in real-world applications [3].

To address these challenges, this project proposes a hybrid intrusion detection system that combines multiple machine learning techniques to improve detection performance. The system integrates Isolation Forest for anomaly detection, Random Forest for classification, and Kullback–Leibler (KL) Divergence for statistical analysis of traffic behavior. These components are combined using a stacking approach to produce a robust and adaptive detection mechanism.

Furthermore, a dynamic thresholding method is introduced to enhance the system's adaptability to changing traffic patterns. The proposed system is designed to operate in a near real-time environment, enabling continuous monitoring and detection of potential threats in MQTT-based IoT networks.

The main objective of this work is to develop an intelligent and adaptive intrusion detection system capable of achieving high detection accuracy while maintaining a low false positive rate. The system aims to provide a practical solution for securing IoT environments against emerging cyber threats.

Problem Statement

Despite the widespread adoption of IoT technologies and MQTT protocol, existing intrusion detection systems face significant challenges in effectively securing such environments. Traditional IDS approaches often rely on static rules or signatures, making them ineffective against evolving and unknown attack patterns.

Additionally, many existing machine learning-based solutions focus on either classification or anomaly detection, but fail to combine both approaches effectively. This results in limitations such as high false positive rates, lack of adaptability, and inability to detect both known and unknown threats efficiently.

In particular, there is a lack of intrusion detection systems that integrate supervised classification, anomaly detection, and statistical analysis within a unified adaptive framework. This limitation reduces the ability of existing systems to effectively detect both known and unknown attacks while maintaining low false positive rates in dynamic IoT environments.

Objectives

The main objectives of this project are as follows:

- Design a hybrid intrusion detection system for MQTT-based IoT networks.
- Integrate supervised, unsupervised, and statistical detection techniques into a unified framework.
- Improve detection accuracy while minimizing false positive rates.
- Develop a dynamic thresholding mechanism for adaptive decision-making.
- Implement a near real-time system using chunk-based data processing.

Chapter 1 — Theoretical Background

This chapter presents the theoretical foundations relevant to the proposed intrusion detection system. It provides an overview of key concepts and technologies used in the project, including the Internet of Things (IoT), the MQTT communication protocol, intrusion detection systems (IDS), and machine learning techniques.

The purpose of this chapter is to establish a clear understanding of the underlying principles that support the design and implementation of the proposed system. In addition, it highlights the challenges associated with securing IoT environments and the limitations of traditional detection methods.

By reviewing these concepts, this chapter lays the groundwork for the proposed hybrid approach, which integrates multiple detection techniques to improve accuracy, adaptability, and overall system performance.

1.1 Internet of Things (IoT)

The Internet of Things (IoT) refers to a network of interconnected devices that communicate and exchange data over the Internet without requiring human intervention. These devices include sensors, actuators, and smart systems deployed in various domains such as healthcare, smart homes, and industrial automation. The rapid growth of IoT has enabled efficient data collection and real-time decision-making, significantly improving system performance and user experience [4].

However, IoT devices are typically resource-constrained, meaning they have limited processing power, memory, and energy capacity. This limitation makes it difficult to implement complex security mechanisms, thereby increasing the vulnerability of IoT environments to cyber threats. As a result, ensuring secure communication and anomaly detection in IoT networks has become a critical research challenge [5].

1.2 MQTT Protocol

The Message Queuing Telemetry Transport (MQTT) protocol is a lightweight messaging protocol designed for resource-constrained devices and low-bandwidth networks. It follows a publish/subscribe communication model, where clients communicate through a central broker

that manages message distribution. This architecture allows efficient and scalable data exchange, making MQTT widely used in IoT applications [6].

In the MQTT model, publishers send messages to specific topics, while subscribers receive messages from topics of interest through the broker. This decoupled communication mechanism improves flexibility and reduces network overhead. However, the simplicity of MQTT also introduces security challenges, as it lacks strong built-in authentication and encryption mechanisms by default [7].

As a result, MQTT-based systems are vulnerable to various attacks, including message flooding, unauthorized access, and denial-of-service attacks. These vulnerabilities highlight the need for intelligent intrusion detection systems capable of monitoring MQTT traffic and identifying abnormal behavior patterns [8].

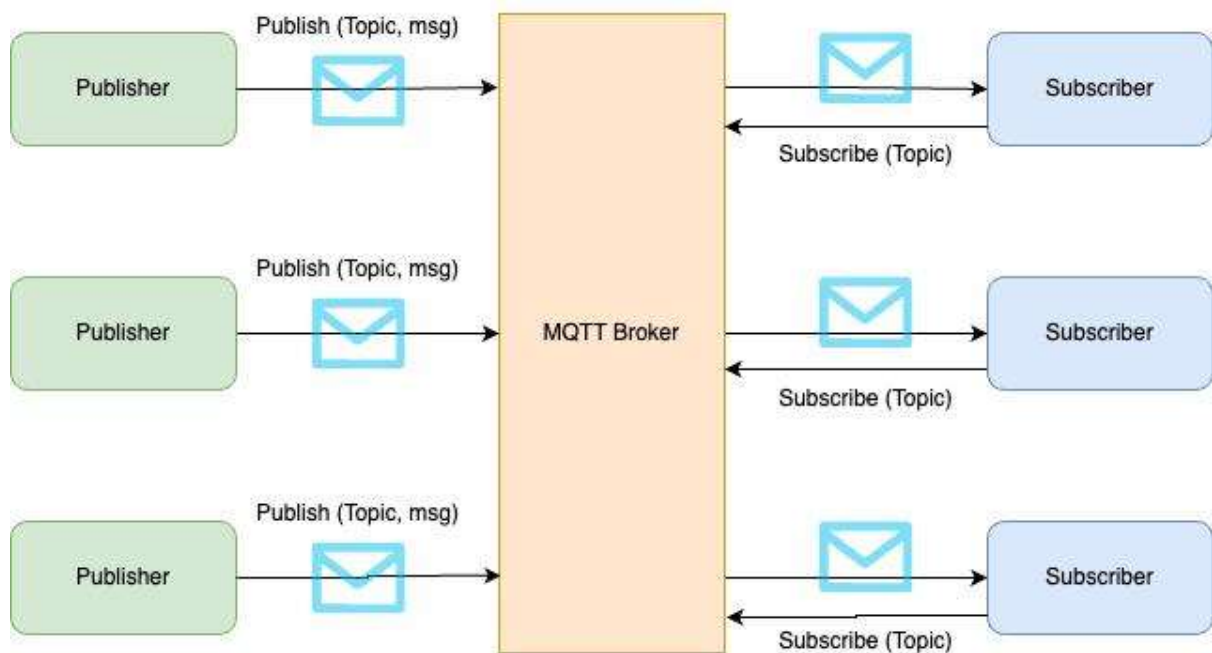


Figure 1 : MQTT publish/subscribe communication model.

As shown in Figure 1, MQTT relies on a broker-based architecture that enables efficient communication between publishers and subscribers.

1.3 Intrusion Detection Systems (IDS)

Intrusion Detection Systems (IDS) are security mechanisms designed to monitor network traffic and identify malicious activities or policy violations. IDS play a critical role in protecting network infrastructures by detecting potential attacks and generating alerts for further analysis.

In IoT environments, IDS are particularly important due to the increased exposure of connected devices to cyber threats [9].

IDS can be broadly classified into two main categories: signature-based detection and anomaly-based detection. Signature-based systems rely on predefined attack patterns to identify known threats, making them effective for detecting previously observed attacks. However, they are unable to detect new or unknown attacks. On the other hand, anomaly-based systems model normal behavior and detect deviations from it, enabling the identification of novel and emerging threats [10].

Despite their advantages, traditional IDS face several limitations in IoT environments. These include high false positive rates, limited adaptability to dynamic traffic patterns, and inefficiency in handling large-scale data generated by IoT devices. These challenges highlight the need for advanced and intelligent IDS solutions that leverage machine learning techniques to improve detection accuracy and adaptability [11].

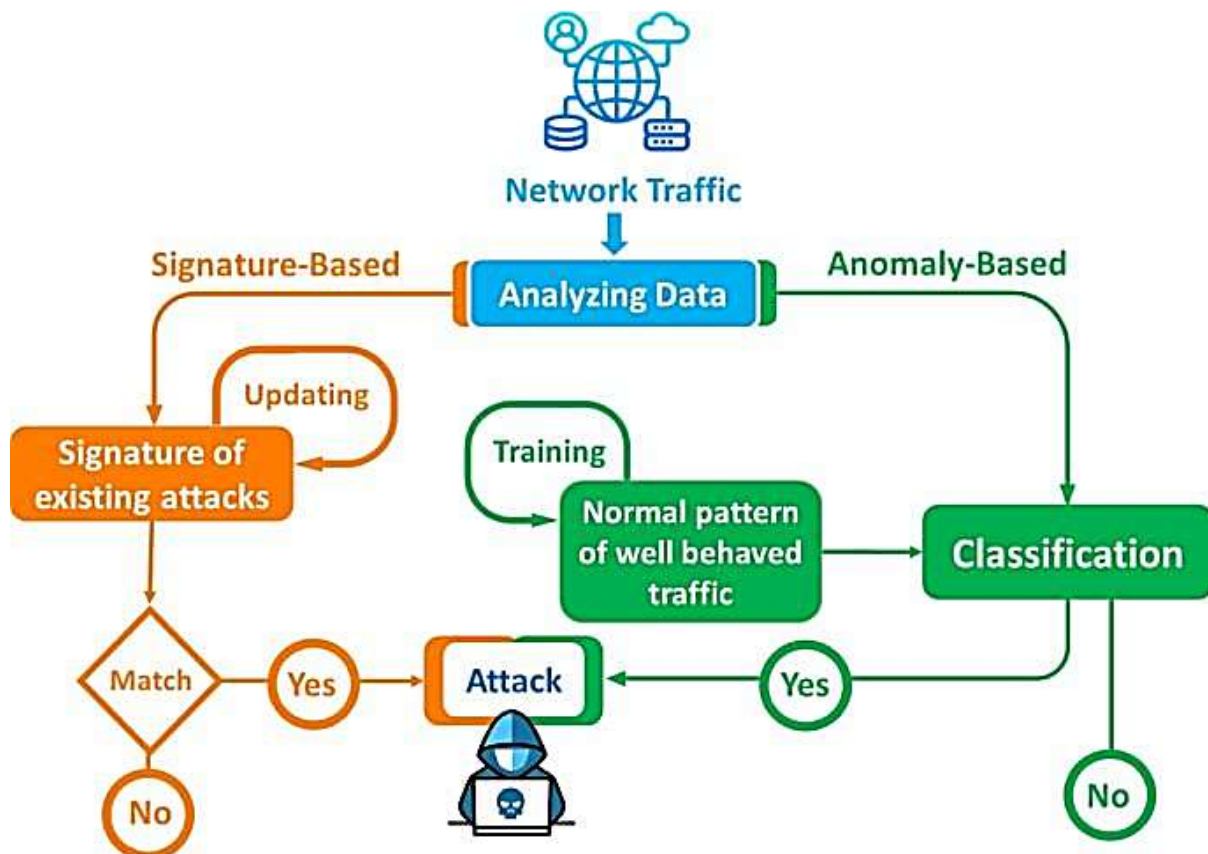


Figure 2 : Classification of intrusion detection systems.

As illustrated in Figure 2, intrusion detection systems can be categorized into signature-based and anomaly-based approaches.

1.4 Machine Learning in Intrusion Detection

Machine learning (ML) has emerged as a powerful approach for enhancing intrusion detection systems, particularly in dynamic environments such as IoT networks. Unlike traditional rule-based methods, ML-based systems can learn patterns from data and adapt to new and evolving threats. This capability makes machine learning highly suitable for detecting both known and unknown attacks in network traffic [12].

Supervised learning algorithms, such as Random Forest, are commonly used for classification tasks in IDS, where labeled data is available to distinguish between normal and malicious traffic. These models can achieve high accuracy but may struggle to generalize to unseen attack patterns. In contrast, unsupervised learning techniques, such as Isolation Forest, are used to detect anomalies by identifying data points that significantly differ from normal behavior without requiring labeled data [13].

In addition to these approaches, statistical methods such as Kullback–Leibler (KL) Divergence are used to measure the deviation between probability distributions. This allows the system to detect shifts in traffic behavior at a global level, complementing the local detection capabilities of machine learning models. By combining supervised, unsupervised, and statistical techniques, hybrid models can provide more robust and adaptive intrusion detection solutions [14].

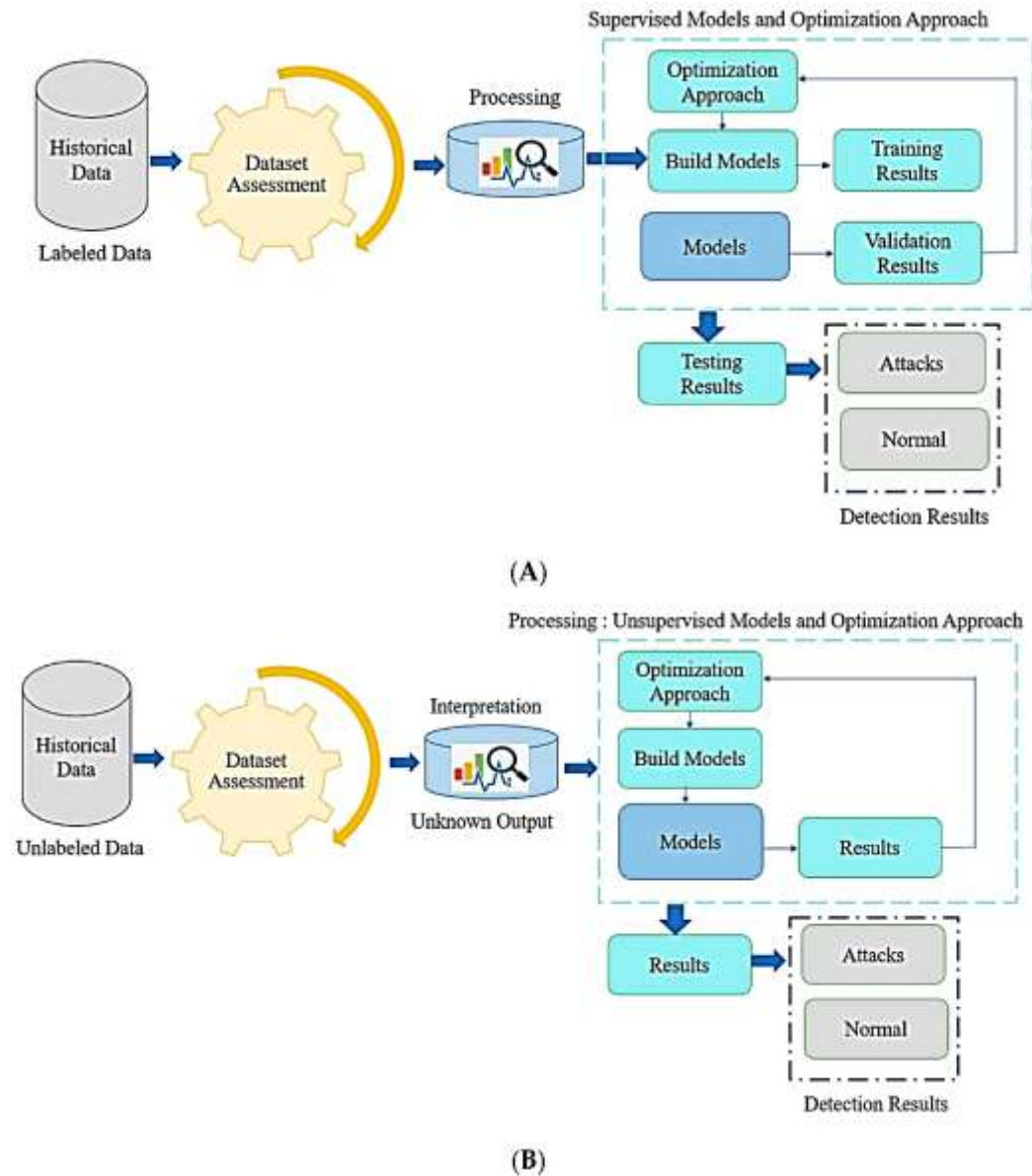


Figure 3 Supervised and Unsupervised Machine Learning Workflows for Intrusion Detection Systems.

Figure 3 illustrates the workflow of both supervised and unsupervised machine learning approaches in intrusion detection systems. In the supervised approach (A), labeled data is used

to train models, allowing the system to learn patterns associated with normal and malicious traffic. The trained models are then validated and tested to produce classification results.

In contrast, the unsupervised approach (B) operates on unlabeled data, where the system identifies anomalies by detecting deviations from normal behavior without prior knowledge of attack patterns. This method is particularly useful for discovering unknown or emerging threats.

The comparison highlights the strengths of both approaches, where supervised models provide accurate classification for known attacks, while unsupervised models enhance the ability to detect novel anomalies. This supports the motivation for adopting hybrid approaches in intrusion detection systems.

This concept is aligned with the proposed system, which combines both supervised and unsupervised techniques to improve detection performance.

This chapter presented the theoretical foundations related to IoT security, MQTT protocol vulnerabilities, intrusion detection systems, and machine learning techniques used in cybersecurity.

The discussion highlighted the limitations of traditional IDS approaches in dynamic IoT environments and emphasized the importance of hybrid and adaptive detection mechanisms. These concepts provide the theoretical basis for the proposed intrusion detection system presented in the following chapters.

Chapter 2 — Related Work

This chapter reviews recent research studies related to intrusion detection systems in IoT environments, with a particular focus on MQTT-based networks and machine learning approaches. The aim is to analyze existing solutions, identify their strengths and limitations, and highlight the research gaps that motivate the proposed work.

Several studies have explored the use of machine learning and hybrid models for detecting cyber-attacks in IoT systems. These approaches vary in terms of techniques, datasets, and evaluation methods. This chapter provides a comparative overview of these studies to position the proposed system within the current state of the art.

Table 1 presents a comparative analysis of recent intrusion detection approaches in IoT environments.

Table 1 : Comparison of Related Work in IoT Intrusion Detection Systems

Ref	Authors	Paper Title	Year	Approach	Contribution	Relation to This Work
[1]	A. Alshamrani et al.	A Survey on Advanced Persistent Threats	2023	ML-based IDS	Comprehensive overview of IDS techniques	Highlights need for adaptive IDS in dynamic environments
[13]	F. T. Liu et al.	Isolation Forest for Anomaly Detection	2024	Unsupervised	Effective anomaly detection method	Used in our system for detecting abnormal behavior
[10]	A. Khraisat et al.	Survey of Intrusion Detection Systems	2024	Hybrid IDS	Comparison between IDS types	Supports adoption of hybrid detection approach
[7]	J. Lin et al.	IoT Security and Privacy Survey	2024	Deep Learning	High detection accuracy	Shows limitations of heavy models in IoT
[14]	T. M. Cover & J. Thomas	Elements of Information Theory	2023	Statistical	KL divergence for distribution comparison	Used for detecting behavior deviation
[20]	Y. Meidan et al.	N-BaIoT: IoT Botnet Detection	2023	Deep Learning	Detects IoT botnet attacks	Shows need for

						lightweight alternatives
[21]	M. Ammar et al.	Security of IoT Frameworks	2023	ML-based	Analysis of IoT security challenges	Justifies the research problem
[22]	S. Verma et al.	Lightweight ML for IoT Security	2025	Lightweight ML	Efficient detection with low resources	Supports lightweight model design
[23]	H. Zhou et al.	Hybrid Intrusion Detection Model	2024	Hybrid	Combines anomaly + classification	Similar to our hybrid architecture
[24]	K. Singh et al.	Adaptive IDS with Dynamic Threshold	2025	Adaptive IDS	Dynamic threshold-based detection	Supports our adaptive threshold mechanism

From the comparison above, it can be observed that existing approaches either focus on accuracy, anomaly detection, or adaptability, but rarely combine all aspects together. This highlights a research gap in developing a hybrid, adaptive, and real-time intrusion detection system, which motivates the proposed work in this project.

This chapter reviewed recent research related to intrusion detection systems in IoT environments and analyzed different machine learning and hybrid detection approaches.

The comparison demonstrated that existing solutions often focus on specific aspects such as classification accuracy, anomaly detection, or adaptability, but rarely combine all these capabilities within a unified framework. This analysis highlights the need for a hybrid and adaptive intrusion detection system, motivating the proposed approach in this work.

Chapter 3 — Proposed System

This chapter presents the proposed intrusion detection system designed for MQTT-based IoT networks. The system aims to address the limitations of traditional intrusion detection approaches by combining multiple machine learning techniques with statistical analysis and adaptive decision-making.

The proposed system follows a hybrid approach that integrates supervised learning, unsupervised learning, and statistical methods to improve detection accuracy and robustness. Specifically, it utilizes Random Forest for classification, Isolation Forest for anomaly detection, and Kullback–Leibler (KL) Divergence for measuring deviations in traffic behavior.

In addition, a dynamic thresholding mechanism is introduced to adapt the detection process to changing network conditions. The system is designed to operate in a near real-time environment, where data is processed in chunks and analyzed continuously.

This chapter describes the architecture of the proposed system, the data processing pipeline, the feature engineering process, and the mathematical model used for decision-making.

3.1 System Architecture

The proposed system architecture is designed to provide an efficient and adaptive intrusion detection mechanism for MQTT-based IoT networks. The architecture consists of multiple interconnected components that work together to process incoming network traffic, extract meaningful features, and perform intelligent decision-making.

The system begins with the data acquisition module, where raw network traffic is collected and organized into manageable chunks. This is followed by the data preprocessing stage, which includes cleaning, normalization, and transformation of the data to ensure consistency and quality.

Next, the feature engineering module extracts relevant features that capture protocol behavior, temporal patterns, and interaction characteristics. These features are then passed to the hybrid detection module, which combines multiple models including Random Forest for classification, Isolation Forest for anomaly detection, and KL Divergence for statistical analysis.

The outputs of these models are integrated using a stacking mechanism to produce a final anomaly score. This score is then evaluated against a dynamic threshold that adapts to the distribution of incoming data. Based on this comparison, the system classifies traffic into normal, suspicious, or high-risk categories.

Finally, the results are visualized through an interactive dashboard that provides real-time insights into system behavior, including anomaly scores, threshold values, and detected attacks. The overall architecture of the proposed system is shown in Figure 4.

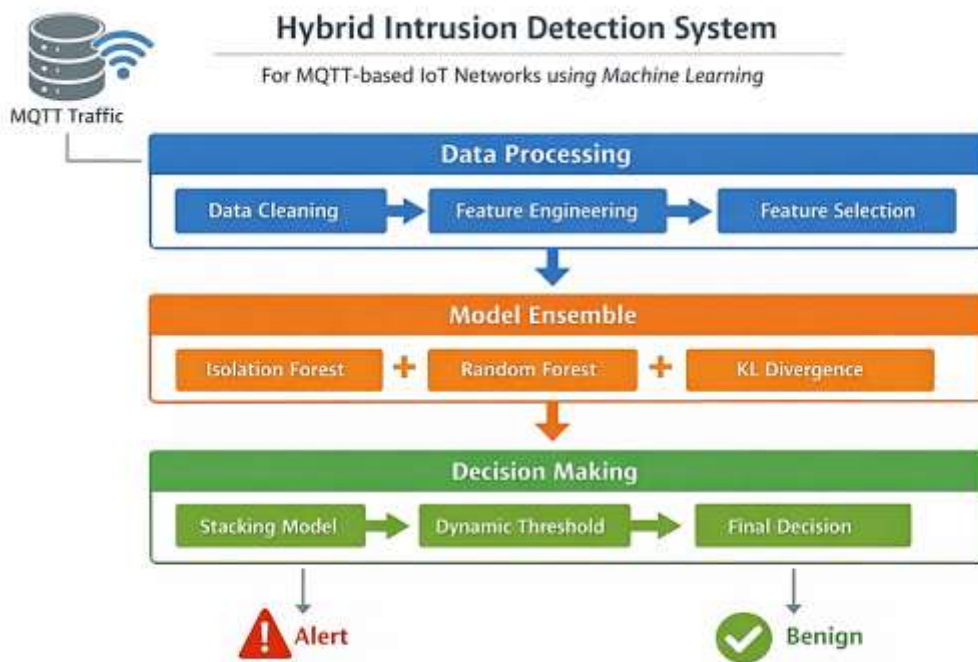


Figure 4 : Proposed hybrid intrusion detection system architecture for MQTT-based IoT networks.

Figure 4 illustrates the architecture of the proposed hybrid intrusion detection system. The system begins with MQTT traffic input, which is processed through multiple stages. In the data processing phase, raw data undergoes cleaning, feature engineering, and feature selection to extract meaningful information.

The processed data is then passed to the model ensemble layer, which combines three main components: Isolation Forest for anomaly detection, Random Forest for classification, and KL Divergence for measuring statistical deviation from normal behavior.

The outputs of these models are integrated using a stacking approach in the decision-making layer. A dynamic threshold is applied to adaptively determine whether the traffic is normal or

anomalous. Based on this process, the system produces the final decision, classifying traffic as either benign or malicious and generating alerts when necessary.

3.2 Data Processing Pipeline

The data processing pipeline is a critical component of the proposed system, responsible for preparing raw network traffic data for analysis and model input. The pipeline is designed to handle large-scale IoT traffic efficiently while maintaining data quality and consistency.

The process begins with loading the dataset, where network traffic is divided into smaller chunks to enable near real-time processing and reduce computational overhead. This chunk-based approach allows the system to process data incrementally and continuously.

Next, the data cleaning stage is applied to remove missing values, inconsistent entries, and irrelevant information. This step ensures that the dataset is reliable and suitable for further processing.

After cleaning, feature engineering is performed to extract meaningful characteristics from the raw data. These features include protocol-specific attributes (such as MQTT header flags and message length), temporal features (such as time differences between packets), and behavioral features (such as message rate and frequency patterns).

Following feature extraction, feature selection techniques are applied to identify the most relevant features and eliminate redundant or less informative ones. This helps improve model performance and reduces computational complexity.

Overall, the data processing pipeline transforms raw IoT traffic into structured and informative data that can be effectively used by the hybrid detection model.

3.3 Feature Engineering

Feature engineering is a crucial step in the proposed system, as it transforms raw network traffic data into meaningful representations that can be effectively used for intrusion detection. The goal of this process is to capture patterns related to protocol behavior, temporal dynamics, and traffic characteristics.

In this work, a set of features was carefully selected to represent different aspects of MQTT-based IoT traffic. These features include protocol-level attributes such as MQTT header flags and message identifiers, traffic-based features such as packet size and message rate, and temporal features such as time differences between packets. In addition, behavioral features were derived to capture interaction patterns and frequency distributions within the traffic.

The selection of features was not arbitrary but based on three main factors: domain knowledge of MQTT protocol behavior, insights from related work, and experimental evaluation. Feature selection techniques were applied to identify the most relevant subset of features and eliminate redundant or less informative ones. Experimental results showed that selecting 12 features provided better performance compared to using the full feature set, improving model generalization and reducing noise.

The selected features used in this work are listed in Table 2.

Table 2 presents the mapping between the selected features and the supporting literature.

Table 2 Mapping of Selected Features to Supporting Research

Feature	Description	Supported by Literature	Purpose in This Work
mqtt.hdrflags	MQTT header flags	[23]	Identifies abnormal MQTT control behavior
mqtt.len	MQTT message length	[23]	Detects unusual message sizes
tcp.len	TCP packet size	[11]	Captures traffic anomalies
mqtt.msgid	MQTT message identifier	[23]	Tracks message patterns and irregularities
msg_rate	Message rate per time unit	[11]	Detects flooding and DDoS attacks
tcp.time_delta	Time difference between packets	[16]	Captures temporal anomalies
qos_freq	QoS frequency distribution	[23]	Identifies abnormal QoS usage

msgtype_freq	Message type frequency	[23]	Detects abnormal protocol behavior
fast_packet	High-speed packet indicator	[11]	Detects rapid traffic bursts
tcp.flags	TCP flags behavior	[16]	Identifies suspicious TCP patterns
rolling_msgtype_mean	Moving average of message types	[17]	Captures behavioral trends
dup_msg_interaction	Duplicate message interaction	[23]	Detects replay or abnormal interactions

As shown in Table 2, the selected features are supported by previous research and are designed to capture different dimensions of network behavior, including protocol activity, traffic intensity, and temporal patterns. This combination of features enhances the ability of the system to detect both known and unknown attacks effectively.

Feature Selection Strategy

The selection of features in this work is based on a systematic and data-driven approach rather than arbitrary choice. The objective is to identify a subset of features that maximizes detection performance while reducing redundancy and computational complexity.

Initially, a larger set of features was extracted from the dataset, capturing different aspects of network traffic behavior, including protocol-level attributes, temporal patterns, and statistical characteristics.

To refine this set, a multi-step feature selection process was applied:

- Low-variance filtering was used to remove features that do not provide meaningful variation across samples.
- Correlation analysis was applied to eliminate highly correlated features, reducing redundancy and preventing model bias.

- Mutual Information (MI) was used to measure the dependency between each feature and the target label, allowing the selection of features that contribute the most to classification performance.

Based on this process, a subset of 12 features was selected from the original feature space. This subset provides a balance between model performance and computational efficiency.

Justification of Feature Selection

The selected features are designed to capture different dimensions of network behavior:

- Protocol-level features (e.g., `mqtt.hdrflags`, `mqtt.msgid`) capture abnormal MQTT communication patterns.
- Traffic characteristics (e.g., `tcp.len`, `msg_rate`) reflect changes in packet size and transmission behavior.
- Temporal features (e.g., `tcp.time_delta`) capture irregular timing patterns associated with attacks.
- Behavioral features (e.g., `qos_freq`, `msgtype_freq`, `rolling_msgtype_mean`) describe long-term traffic behavior and distribution.
- Interaction-based features (e.g., `dup_msg_interaction`) help identify replay or abnormal communication sequences.

This combination ensures that the model can detect both local anomalies and global behavioral changes.

Relation to Existing Work

The feature selection approach is consistent with prior research in IoT intrusion detection, where a combination of protocol-specific, statistical, and temporal features is commonly used to improve detection performance.

In particular:

- MQTT-related features are supported by prior studies focusing on protocol-level anomaly detection [23].
- Traffic and statistical features are widely used in IoT intrusion detection datasets such as N-BaIoT [11]
- Temporal and behavioral features are emphasized in recent works on adaptive intrusion detection and concept drift handling [17], [19].

Experimental Validation

The effectiveness of the selected feature set was validated experimentally by evaluating model performance during training and testing. Results showed that reducing the feature set to 12 features improved model efficiency while maintaining high detection accuracy.

This demonstrates that the selected features are both relevant and sufficient for the proposed hybrid detection system.

3.4 Hybrid Detection Model

The proposed intrusion detection system is based on a hybrid model that integrates multiple detection techniques to improve accuracy, robustness, and adaptability. Instead of relying on a single model, the system combines supervised learning, unsupervised learning, and statistical analysis to capture different aspects of network behavior.

The hybrid model consists of three main components: Random Forest (RF), Isolation Forest (IF), and Kullback–Leibler (KL) Divergence. Each component plays a distinct role in the detection process.

Random Forest is used as a supervised learning model to classify network traffic into normal or malicious categories based on labeled data. It is effective in capturing complex patterns and relationships between features, providing high classification accuracy for known attack types.

Isolation Forest is an unsupervised anomaly detection algorithm that identifies abnormal data points by isolating them in a feature space. Unlike supervised models, it does not require labeled data and is particularly useful for detecting unknown or novel attacks.

KL Divergence is employed as a statistical measure to quantify the difference between the current traffic distribution and a reference distribution representing normal behavior. This allows the system to detect global deviations in network behavior that may not be captured by individual models.

The outputs of these three components are combined using a stacking approach. Each model produces a score representing the likelihood of anomalous behavior. These scores are then aggregated to form a final decision score, which reflects the overall risk level associated with the input data.

This hybrid approach enhances detection performance by leveraging the strengths of each method. Random Forest improves classification accuracy, Isolation Forest enhances anomaly detection, and KL Divergence captures distribution-level changes. Together, they provide a comprehensive and adaptive intrusion detection mechanism suitable for dynamic IoT environments.

The integration of multiple detection techniques reduces the limitations associated with individual models. While supervised models may fail to detect unseen attacks, and unsupervised models may produce higher false positives, the hybrid approach balances these trade-offs to achieve more reliable and stable detection results.

3.5 Dynamic Threshold

The decision threshold in the proposed system is not fixed, but dynamically adjusted to adapt to changes in network traffic behavior. This is particularly important in IoT environments, where data distributions are non-stationary and subject to concept drift.

Let $S = \{s_1, s_2, \dots, s_n\}$ denote the set of anomaly scores produced by the stacking model for a given data chunk. The dynamic threshold T is computed based on the statistical distribution of these scores using a percentile-based approach:

$$T = \text{Percentile}(S, 100 \times (1 - r))$$

where $r \in (0,1)$ represents the expected proportion of anomalous samples in the data.

This formulation ensures that only the highest anomaly scores are classified as attacks, making the threshold adaptive to variations in score distribution across different time intervals.

Threshold Smoothing using EMA

To avoid instability and sudden fluctuations in the threshold values between consecutive chunks, an Exponential Moving Average (EMA) is applied:

$$T_t = \alpha \cdot T_{\text{current}} + (1 - \alpha) \cdot T_{t-1}$$

where:

- T_t is the updated threshold at time t
- T_{current} is the threshold computed from the current chunk
- T_{t-1} is the previous threshold
- $\alpha \in (0,1)$ is the smoothing factor

The EMA reduces variance in threshold updates and provides temporal stability, ensuring consistent detection performance.

Justification

Static thresholds are not suitable for IoT environments due to dynamic traffic patterns and evolving attack behaviors. In contrast, percentile-based thresholding adapts to the distribution of anomaly scores, making it robust to changes in data scale and variability.

The use of EMA further enhances stability by smoothing short-term fluctuations, which is particularly important in streaming or near real-time scenarios.

This approach is consistent with prior work in anomaly detection and concept drift adaptation, where adaptive thresholding techniques are used to maintain detection performance under non-stationary conditions [15], [19], [22].

Interpretation

The dynamic threshold enables the system to:

- Automatically adjust sensitivity based on traffic conditions
- Reduce false positives during normal traffic periods
- Increase detection sensitivity during abnormal activity
- Maintain stability across consecutive data chunks

This makes the system more suitable for deployment in real-world IoT environments compared to traditional fixed-threshold approaches.

Unlike static threshold approaches, the proposed adaptive mechanism continuously responds to changes in traffic behavior and anomaly score distributions. This improves robustness under fluctuating network conditions and reduces the risk of performance degradation caused by concept drift.

The adaptive threshold mechanism enables the system to dynamically balance detection sensitivity and false positive control without requiring manual threshold tuning. This

characteristic makes the proposed approach more suitable for practical IoT environments, where traffic behavior is highly dynamic and unpredictable.

The behavior of the dynamic threshold over time is illustrated in Figure 5.

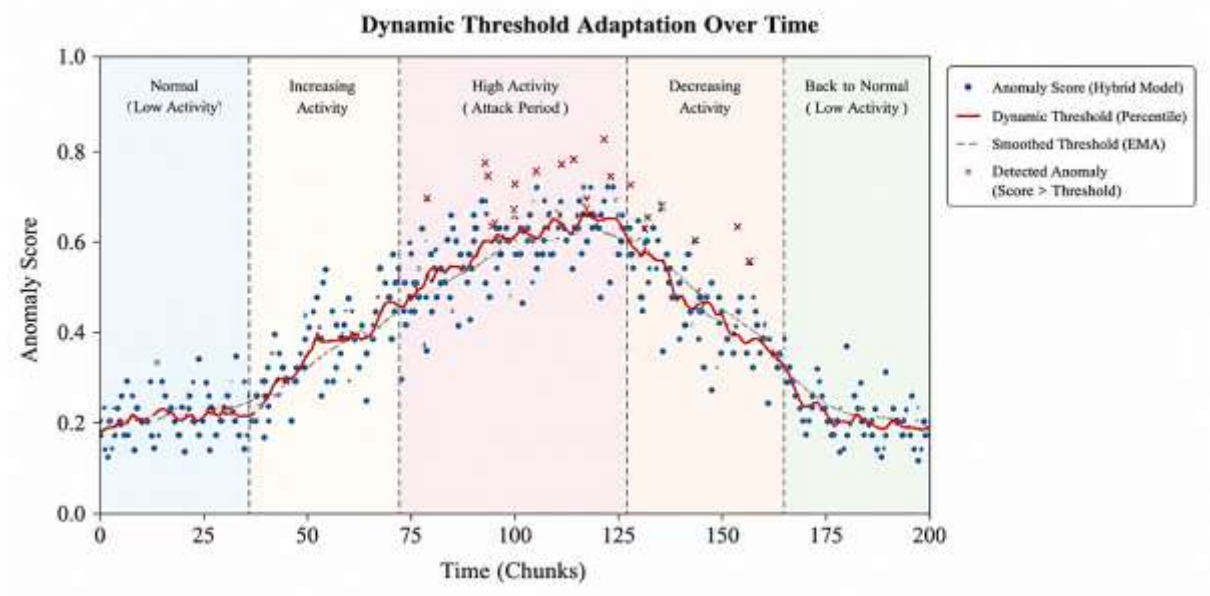


Figure 5: Dynamic threshold adaptation over time based on anomaly score distribution.

Figure 5 shows the variation of anomaly scores over time along with the dynamically computed threshold. The threshold adapts to changes in the distribution of the scores, increasing during periods of high activity and decreasing when the traffic returns to normal. This adaptive behavior allows the system to maintain stable and reliable detection performance in dynamic IoT environments.

3.6 Mathematical Model

The proposed intrusion detection system is formulated as a hybrid mathematical model that integrates supervised, unsupervised, and statistical detection mechanisms into a unified decision function.

Let $x \in \mathbb{R}^d$ represent a feature vector extracted from MQTT network traffic. The system applies three distinct detection components, each producing a score that reflects a different aspect of network behavior.

1) Random Forest (RF)

The Random Forest model produces a classification probability representing the likelihood that a sample belongs to the attack class:

$$RF(x) = P(y = 1 \mid x)$$

where $y \in \{0,1\}$ denotes normal or attack traffic. This probability is obtained by averaging the outputs of multiple decision trees:

$$RF(x) = (1 / N) \sum_{i=1}^N T_i(x)$$

2) Isolation Forest (IF)

The Isolation Forest assigns an anomaly score based on how easily a data point can be isolated:

$$IF(x) = 2^{\{- E(h(x)) / c(n)\}}$$

where $E(h(x))$ is the expected path length required to isolate x , and $c(n)$ is a normalization factor dependent on the dataset size.

3) KL Divergence

The KL Divergence measures the deviation between the current traffic distribution P and a reference distribution Q representing normal behavior:

$$KL(x) = D_{\{KL\}}(P \parallel Q) = \sum P(x) \log (P(x) / Q(x))$$

This component captures global changes in traffic behavior that may not be detected by instance-based models.

Score Normalization

Since the outputs of the three models are on different scales, a normalization step is applied to ensure comparability:

RF_norm(x), IF_norm(x), KL_norm(x)

Each score is transformed into a common range (e.g., [0,1]) using min-max normalization:

$$s_{\text{norm}} = (s - s_{\text{min}}) / (s_{\text{max}} - s_{\text{min}})$$

Stacking Model (Meta-Learner)

The normalized scores are combined using a stacking-based meta-model:

$$S(x) = M(\text{RF_norm}(x), \text{IF_norm}(x), \text{KL_norm}(x))$$

where M is a Logistic Regression model trained to learn the optimal combination of the input scores.

The use of Logistic Regression provides a probabilistic interpretation of the final decision and ensures stable and interpretable integration of multiple detection signals.

Final Decision Function

The final anomaly score $S(x)$ is compared against a dynamic threshold T :

If $S(x) > T \rightarrow \text{Attack}$

If $S(x) \leq T \rightarrow \text{Normal}$

The threshold T is computed adaptively based on the distribution of anomaly scores using percentile analysis and smoothed using Exponential Moving Average (EMA).

Model Interpretation

The proposed formulation enables the system to:

- Capture local patterns using Random Forest
- Detect isolated anomalies using Isolation Forest
- Identify global distribution shifts using KL Divergence

The stacking model integrates these complementary perspectives into a unified and data-driven decision function, improving robustness and detection accuracy in dynamic IoT environments.

3.7 Real-Time Implementation

The proposed system is implemented in a near real-time environment to continuously monitor and analyze MQTT network traffic. The implementation is designed to process incoming data streams efficiently while maintaining low latency and high detection performance.

The system operates using a chunk-based processing approach, where incoming data is divided into smaller segments. Each chunk is processed independently through the data pipeline, including preprocessing, feature extraction, and hybrid model evaluation. This approach allows the system to handle continuous data streams without overloading computational resources.

The detection process is performed sequentially for each chunk. First, anomaly scores are generated using the hybrid model, which combines Random Forest, Isolation Forest, and KL Divergence. Then, the dynamic threshold is computed based on the distribution of the scores within the current chunk. Finally, each data point is classified as normal or anomalous based on its score relative to the threshold.

The system is integrated with a Streamlit-based interface that provides real-time visualization of detection results. The interface displays anomaly scores, threshold values, detected attacks,

and overall system status. This allows users to monitor network behavior and respond to potential threats effectively.

The use of real-time processing and interactive visualization enhances the practical applicability of the system in IoT environments, where continuous monitoring and rapid response are essential.

The chunk-based processing strategy improves system scalability and enables adaptive analysis of streaming data. It also allows the dynamic threshold to be updated continuously, ensuring that the detection mechanism remains responsive to changes in network behavior.

This chapter presented the architecture and design of the proposed hybrid intrusion detection system for MQTT-based IoT networks.

The chapter described the data processing pipeline, feature engineering strategy, hybrid detection model, dynamic threshold mechanism, mathematical formulation, and near real-time implementation. Together, these components form an adaptive and robust framework for detecting both known and unknown attacks in dynamic IoT environments.

Chapter 4 — Implementation and Experimental Setup

4.1 Dataset Description

This section describes the dataset used to train and evaluate the proposed intrusion detection system. The experiments in this work are conducted using the MQTTset dataset, which is specifically designed for intrusion detection in MQTT-based IoT environments.

The MQTTset dataset contains network traffic data collected from IoT systems using the MQTT protocol. It includes both legitimate traffic and various types of cyber-attacks, such as denial-of-service (DoS), brute-force attacks, malformed packets, slow-rate attacks, and flooding attacks. The dataset provides labeled instances that enable both supervised and unsupervised learning approaches.

The dataset used in this work consists of a total of 2,097,150 samples. These samples are equally divided into training and testing subsets, where each subset contains 1,048,575 samples.

In terms of class distribution, the training dataset includes 1,034,228 legitimate samples and 14,347 attack samples. Similarly, the testing dataset contains 1,034,103 legitimate samples and 14,472 attack samples.

The dataset is highly imbalanced, with attack samples representing approximately 1.08% of the training data and 1.09% of the testing data. This imbalance reflects realistic IoT network conditions, where malicious traffic is relatively rare compared to normal behavior.

The dataset originally contains multiple features related to network traffic and MQTT protocol behavior. In this work, a subset of 12 relevant features is selected based on feature engineering and selection techniques to improve detection performance and reduce computational complexity.

The use of the MQTTset dataset provides a realistic and representative environment for evaluating the proposed intrusion detection system, as it captures real-world IoT communication patterns and attack scenarios.

The distribution of the dataset used in this work is summarized in Table 3.

Table 3: Dataset Distribution of MQTTset

Dataset	Total Samples	Normal Samples	Attack Samples	Attack Percentage
Training Set	1,048,575	1,034,228	14,347	1.08%
Testing Set	1,048,575	1,034,103	14,472	1.09%
Total	2,097,150	2,068,331	28,819	1.37%

Table 3 presents the distribution of normal and attack samples in the MQTTset dataset.

The table shows that the dataset is highly imbalanced, with attack samples representing a small fraction of the total data, which reflects real-world IoT network conditions.

Implementation Environment

This section describes the implementation environment and tools used to develop and evaluate the proposed intrusion detection system.

The system is implemented using Python as the main programming language due to its extensive support for data analysis and machine learning applications. Several libraries and frameworks are utilized to support different components of the system.

Pandas and NumPy are used for data manipulation and numerical operations. Scikit-learn is used to implement machine learning models, including Random Forest, Isolation Forest, and Logistic Regression for the stacking model.

For real-time visualization and user interaction, Streamlit is used to build an interactive dashboard that displays anomaly scores, detection results, and system behavior.

The system is developed and tested on a Windows-based environment. The implementation is designed to handle large-scale data efficiently using chunk-based processing, enabling near real-time analysis.

Model persistence is achieved using the Joblib library, which allows trained models to be saved and loaded efficiently during deployment.

This combination of tools provides a flexible, scalable, and efficient environment for developing and deploying the proposed intrusion detection system.

Table 4 : Implementation Environment and Tools

Component	Tool / Technology
Programming Language	Python
Data Processing	Pandas, NumPy
Machine Learning	Scikit-learn
Visualization	Streamlit
Model Storage	Joblib
Operating System	Windows

Table 4 summarizes the implementation tools and technologies used in this work.

4.2 Data Preparation

This section describes the data preparation process applied to the MQTTset dataset before training the proposed intrusion detection system. The preparation pipeline is designed to ensure data quality, consistency, and suitability for machine learning models.

First, data cleaning is performed using a dedicated cleaning module. This includes removing duplicate records, handling missing values using statistical imputation (median for numerical features and mode for categorical features), and eliminating irrelevant textual fields related to MQTT messages. In addition, categorical features are converted into numerical representations to make them suitable for model input.

To further improve data quality, constant and low-variance features are removed, and outliers are handled using clipping techniques based on percentile thresholds. This ensures that extreme values do not negatively affect model performance.

Feature engineering is then applied to extract meaningful behavioral, temporal, and protocol-level features from the raw data. These include message frequency, packet timing, rolling statistics, and interaction-based features.

Regarding scaling, numerical normalization is selectively applied in specific components of the system. Isolation Forest and the stacking model utilize feature scaling to ensure stable anomaly score computation and effective model integration. However, Random Forest does not require scaling due to its tree-based nature.

Finally, the dataset is processed using a chunk-based strategy, allowing efficient handling of large-scale data and enabling near real-time processing in the proposed system.

To better illustrate the impact of preprocessing and feature selection operations, Table 5 summarizes the progressive reduction of the feature space throughout the preparation pipeline.

Table 5 : Feature Reduction and Selection Process

Processing Stage	Number of Features	Description
Original Dataset	35	Original MQTT and TCP traffic features
After Data Cleaning	35	Data cleaning, duplicate removal, missing value handling, and text column elimination
After Low Variance Filtering	18	Features with insufficient variance were removed
After Correlation Filtering	13	Highly correlated and redundant features were eliminated
Final Selected Features	12	Most informative features selected using Mutual Information

The preprocessing and feature selection pipeline significantly reduces feature redundancy and improves the quality of the input data. Initially, the dataset contains 35 features extracted from MQTT and TCP traffic. Low variance filtering reduces the feature space to 18 features by eliminating features with minimal variation across samples.

Afterward, correlation filtering removes highly correlated features, resulting in 13 remaining features. Finally, Mutual Information is applied to identify the most informative features related to intrusion detection, producing an optimized subset of 12 features used for training the hybrid detection model.

4.3 Model Training

This section describes the training process of the proposed hybrid intrusion detection system. The training pipeline is designed to integrate multiple detection techniques, including supervised learning, unsupervised learning, and statistical analysis.

The training process begins with preparing the dataset after preprocessing and feature selection. The dataset is split into training and validation subsets using stratified sampling to preserve class distribution.

Overview of the Used Detection Models

Before training the proposed hybrid intrusion detection system, it is important to briefly describe the role and functionality of each detection component used in the framework.

Random Forest (RF) is a supervised ensemble learning algorithm based on multiple decision trees. The model performs classification by aggregating the outputs of individual trees using majority voting. Random Forest is widely used in intrusion detection systems due to its high classification accuracy, robustness against overfitting, and ability to handle high-dimensional datasets. In this work, Random Forest is used to classify MQTT traffic into normal and malicious categories using labeled data.

Isolation Forest (IF) is an unsupervised anomaly detection algorithm designed to isolate abnormal data points. Instead of learning attack signatures, the model identifies anomalies based on how easily samples can be separated within randomly generated trees. Isolation Forest is particularly effective in detecting unknown or previously unseen attacks, making it suitable for dynamic IoT environments.

Kullback–Leibler (KL) Divergence is a statistical measure used to quantify the difference between probability distributions. In the proposed system, KL Divergence is used to compare the current traffic distribution with a reference distribution representing normal behavior. This enables the system to detect global behavioral shifts and subtle traffic changes that may not be captured by instance-based machine learning models.

The combination of these three approaches allows the proposed system to integrate classification, anomaly detection, and statistical analysis within a unified hybrid framework.

The Isolation Forest model is trained using only benign samples to learn the normal behavior of the system. This allows the model to detect anomalies by identifying deviations from the learned normal patterns.

Next, the KL Divergence model is fitted on the training data to establish a statistical reference distribution of normal behavior. This enables the system to measure the deviation of incoming data from expected patterns.

The Random Forest model is then trained using labeled data, where normal and attack samples are used to learn classification boundaries. The model is configured with multiple trees and balanced class weights to handle the imbalanced nature of the dataset.

After training the individual models, a stacking approach is used to combine their outputs. The outputs of Random Forest (classification probability), Isolation Forest (anomaly score), and KL Divergence (statistical score) are used as input features to train a meta-model.

A Logistic Regression model is used as the meta-model, which learns how to optimally combine the outputs of the individual models. Before training the meta-model, the input features are normalized using a StandardScaler to ensure consistent scale across inputs.

The final model is evaluated using a validation dataset, where an optimal decision threshold is determined using ROC curve analysis and the Youden Index. This threshold is then used as a reference for classification during testing and real-time deployment.

The use of a hybrid training strategy allows the system to leverage the strengths of different detection techniques. Isolation Forest enhances the detection of unknown anomalies, Random Forest improves classification of known attacks, and KL Divergence captures distribution-level changes. The stacking model integrates these outputs into a unified decision framework, improving overall detection performance and robustness.

4.4 Evaluation Setup

This section describes the evaluation methodology used to assess the performance of the proposed intrusion detection system. The evaluation is designed to measure the effectiveness of the model in detecting attacks while minimizing false positives.

Several performance metrics are used to evaluate the system, including accuracy, precision, recall, F1-score, false positive rate (FPR), and area under the ROC curve (ROC-AUC). These metrics provide a comprehensive understanding of the model's performance.

Among these metrics, the F1-score is considered the most important evaluation measure in this work. This is because the dataset is highly imbalanced, with attack samples representing a small fraction of the total data. In such cases, accuracy alone is not sufficient, as a model can achieve high accuracy by predicting the majority class. The F1-score provides a better balance between precision and recall, making it more suitable for intrusion detection tasks.

Precision measures the proportion of correctly identified attack samples among all predicted attacks, while recall measures the proportion of actual attacks that are correctly detected. The F1-score combines both metrics into a single value, providing a balanced evaluation of detection performance.

The model is evaluated on a separate test dataset that was not used during training. This ensures that the evaluation reflects the model's ability to generalize to unseen data.

Receiver Operating Characteristic (ROC) analysis is used to evaluate the trade-off between the True Positive Rate (TPR) and False Positive Rate (FPR) across different threshold values.

To determine the optimal operating threshold, the Youden Index is applied. The Youden Index identifies the threshold that maximizes the difference between detection sensitivity and false alarm rate, providing a balanced trade-off between attack detection and false positive reduction.

$$J = \text{TPR} - \text{FPR}$$

The threshold corresponding to the maximum Youden Index value is selected as the initial decision threshold used during validation and testing.

This evaluation setup ensures that the proposed system is assessed in a realistic and reliable manner, taking into account both detection capability and false alarm rates.

4.5 System Implementation (Streamlit)

This section presents the implementation of the proposed intrusion detection system using a Streamlit-based interface for real-time monitoring and visualization.

The system is designed to simulate a real-time IoT environment by processing incoming data in chunks. Each data chunk is passed through the full processing pipeline, including data cleaning, feature engineering, feature selection, and hybrid model evaluation.

The implementation integrates previously trained models, including Random Forest, Isolation Forest, KL Divergence, and the stacking meta-model. These models are loaded using the Joblib library and applied directly to incoming data without retraining.

For each data chunk, anomaly scores are computed using the hybrid model. These scores are then normalized and evaluated using a dynamic threshold, which is updated continuously based on the distribution of the current data.

The Streamlit interface provides real-time visualization of the system's behavior. It displays anomaly scores, classification results, threshold values, and alert indicators. The interface also includes graphical representations such as scatter plots for traffic classification and histograms for score distribution.

In addition, key performance indicators (KPIs) are displayed, including detection rate, number of alerts, and current threshold value. These metrics provide a quick overview of system status and allow users to assess the level of threat in the network.

The system continuously updates the display in a loop, simulating streaming data and enabling near real-time intrusion detection. This implementation demonstrates the practical applicability of the proposed system in monitoring IoT network traffic and detecting potential attacks dynamically.

The dynamic threshold is computed using a percentile-based approach and updated using an exponential moving average (EMA) to ensure stability over time. This allows the system to adapt to changes in traffic behavior while avoiding sudden fluctuations in detection decisions. To evaluate the practical applicability of the proposed system in a near real-time environment, a Streamlit-based monitoring dashboard is implemented. The dashboard simulates streaming MQTT traffic using chunk-based processing and continuously updates anomaly scores, detection results, and adaptive threshold values.

The purpose of this experiment is to analyze how the system behaves under changing traffic conditions and to evaluate the effectiveness of the dynamic threshold mechanism in controlling false alarms while maintaining high detection capability.

The real-time monitoring interface of the proposed system is shown in Figure 6.

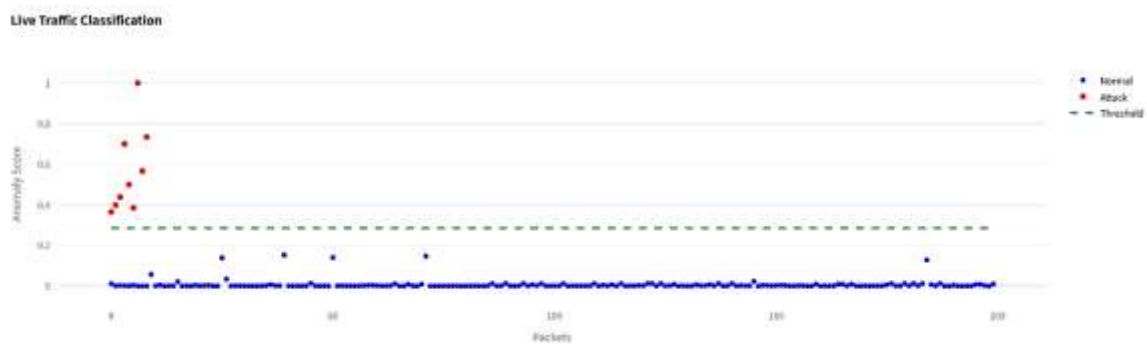


Figure 6: Real-time intrusion detection dashboard implemented using Streamlit.

Figure 6 illustrates the real-time monitoring dashboard implemented using Streamlit. The dashboard visualizes anomaly scores generated by the hybrid model, together with the adaptive threshold used for decision-making.

The plotted points represent individual traffic samples processed within consecutive chunks. Samples exceeding the dynamic threshold are classified as anomalous and highlighted as potential attacks.

The dashboard demonstrates the ability of the proposed system to continuously monitor network behavior, adapt to traffic changes, and provide immediate visual feedback regarding

suspicious activities. This confirms the suitability of the proposed approach for near real-time intrusion detection in MQTT-based IoT environments.

This chapter described the implementation details and experimental setup used to evaluate the proposed intrusion detection system.

The discussion covered dataset preparation, feature selection, model training, evaluation methodology, and the real-time Streamlit implementation. These steps provide the foundation for the experimental results and performance analysis presented in the next chapter.

Chapter 5 — Results and Discussion

5.1 Introduction

This chapter presents the experimental results of the proposed intrusion detection system and provides a detailed analysis of its performance. The evaluation focuses on assessing the system's ability to accurately detect malicious activities in MQTT-based IoT networks while maintaining a low false positive rate.

The results are obtained using the test dataset described in Chapter 4, ensuring that the evaluation reflects the model's generalization capability on unseen data. Various performance metrics are used to provide a comprehensive assessment of the system, with particular emphasis on the F1-score due to the imbalanced nature of the dataset.

It is important to emphasize that the dataset is highly imbalanced, where attack samples represent a small minority of the total data. Therefore, accuracy alone is not sufficient to evaluate performance.

For this reason, greater emphasis is placed on precision, recall, and F1-score, which provide a more reliable assessment of the model's effectiveness in detecting rare attack instances.

Additionally, class weighting was applied in the Random Forest model to reduce bias toward the majority class.

In addition to quantitative results, this chapter discusses the behavior of the system under different conditions, including its ability to adapt to dynamic traffic patterns and detect both known and unknown attacks.

5.2 Quantitative Results

This section presents the quantitative evaluation results of the proposed intrusion detection system on the test dataset. The results demonstrate the effectiveness of the hybrid model in detecting malicious activities in IoT network traffic.

The performance of the system is evaluated using multiple metrics, including accuracy, precision, recall, F1-score, false positive rate (FPR), and ROC-AUC. The results obtained on the test dataset are summarized in Table 7.

The system achieves an accuracy of 99.36%, indicating that the majority of samples are correctly classified. More importantly, the model achieves a precision of 90.20% and a recall of 98.14%, demonstrating its ability to detect a high proportion of attack samples while maintaining a low number of false alarms.

The F1-score reaches 94.00%, which reflects a strong balance between precision and recall. This is particularly important given the imbalanced nature of the dataset, where attack samples represent a small fraction of the total data.

The false positive rate (FPR) is only 0.57%, indicating that the system generates very few false alarms. Additionally, the ROC-AUC score of 0.9969 demonstrates excellent discrimination capability between normal and attack traffic.

Overall, these results confirm that the proposed hybrid model is highly effective in detecting intrusions in MQTT-based IoT environments while maintaining stability and reliability.

To further evaluate the effectiveness of the proposed approach, a comparison with baseline models is presented.

Table 6 Comparison of the Proposed Hybrid Model with Baseline Approaches

Model	Detection Capability	Strengths	Limitations
Random Forest (RF)	Supervised classification	High accuracy for known attacks, interpretable results	Limited ability to detect unknown attacks
Isolation Forest (IF)	Unsupervised anomaly detection	Effective in detecting anomalies without labeled data	Higher false positives in complex traffic patterns
KL Divergence	Statistical detection	Captures global distribution changes and behavioral shifts	Sensitive to noise and requires proper baseline distribution
Hybrid Model (Proposed)	Integrated detection	Combines classification, anomaly detection, and statistical analysis for robust performance	Slightly higher computational complexity

Table 6 compares the proposed hybrid model with individual baseline approaches in terms of detection capability, strengths, and limitations.

The comparison highlights that individual models are effective in specific aspects of intrusion detection but suffer from inherent limitations when used independently. Random Forest performs well in classifying known attack patterns, while Isolation Forest is capable of identifying anomalous behavior without requiring labeled data. KL Divergence captures global shifts in traffic distribution, which are not detected by instance-based models.

However, none of these approaches alone can provide comprehensive detection across all attack scenarios. The proposed hybrid model overcomes these limitations by integrating the strengths of all three components using a stacking-based approach. This results in improved robustness, better generalization, and enhanced detection performance in dynamic IoT environments.

Table 7 : Performance Evaluation Results on Test Dataset

Metric	Value
Accuracy	99.36%
Precision	90.20%
Recall	98.14%
F1-score	94.00%
FPR	0.57%
ROC-AUC	0.9969
PR-AUC	0.9855

The results demonstrate that the proposed hybrid model achieves strong overall performance across multiple evaluation metrics. The system attains an accuracy of 99.36%, indicating that the majority of samples are correctly classified.

The recall reaches 98.14%, showing that the system successfully detects most attack instances. This is particularly important in cybersecurity applications, where failing to detect an attack (false negatives) can have serious consequences. The precision of 90.20% indicates that most detected attacks are indeed malicious, although a small number of normal samples are misclassified as attacks.

The F1-score of 94.00% reflects a good balance between precision and recall, making it a reliable metric for evaluating performance in imbalanced datasets. The low false positive rate (FPR) of 0.57% further confirms that the system generates only a limited number of false alarms, which is essential for maintaining system usability in real-world deployments.

In addition, the high ROC-AUC value of 0.9969 demonstrates excellent discrimination capability between normal and malicious traffic, while the PR-AUC of 0.9855 indicates strong performance in detecting minority attack classes.

Overall, the strong performance of the proposed system can be attributed to its hybrid design. By combining Random Forest, Isolation Forest, and KL Divergence, the system captures different aspects of network behavior. In particular, KL Divergence contributes by detecting global shifts in traffic distribution, enabling the identification of subtle and previously unseen attack patterns. This integration enhances robustness and improves overall detection performance.

To provide a deeper understanding of the model's classification performance, the confusion matrix is presented in terms of percentage values relative to the entire dataset.

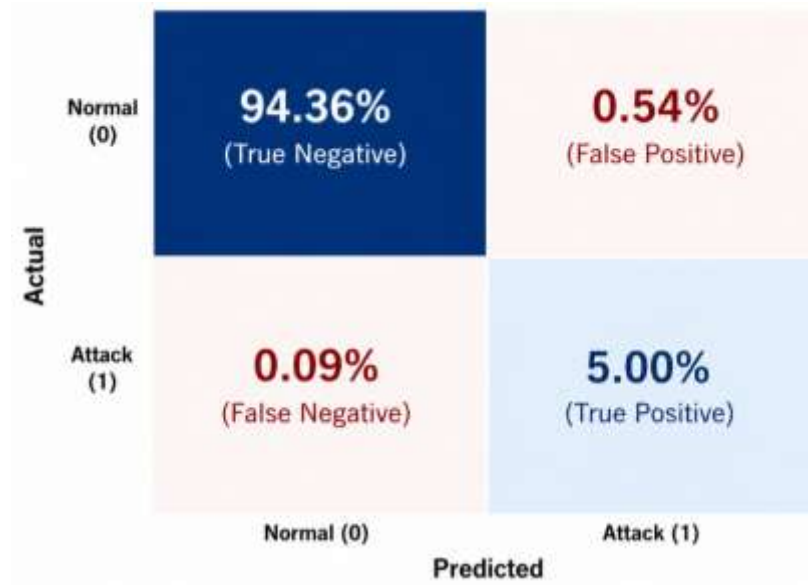


Figure 7: Confusion Matrix (Percentage of Total Dataset)

Figure 7 illustrates the confusion matrix normalized over the entire dataset, where each value represents the percentage of total samples.

As shown in Figure 7, the majority of samples are correctly classified as normal traffic (94.36%), reflecting the dominance of benign data in the dataset. The system correctly identifies 5.00% of samples as attacks, demonstrating strong detection capability.

The false positive rate remains low at 0.54%, indicating that only a small portion of normal traffic is incorrectly classified as malicious. Additionally, the false negative rate is extremely low at 0.09%, which means that very few attack instances are missed.

This distribution highlights the effectiveness of the proposed system in handling imbalanced data, where attack samples represent a small fraction of the total dataset. The results confirm that the model achieves a strong balance between accurate detection and minimizing classification errors.

5.3 Real-Time Results Analysis

This section analyzes the near real-time performance in a simulated streaming environment of the proposed intrusion detection system under streaming conditions. The system processes incoming data using a chunk-based approach, where each chunk contains a subset of network traffic samples.

In this work, a chunk size of 500 samples is used during real-time simulation. This value is selected to balance between computational efficiency and detection responsiveness. Smaller chunk sizes allow faster updates and quicker detection of anomalies, while larger chunk sizes provide more stable statistical estimates for threshold computation. The selected size provides an effective trade-off between these two factors.

For each chunk, the system computes anomaly scores using the hybrid model and updates the dynamic threshold based on the distribution of these scores. The threshold is adjusted using a percentile-based method combined with exponential moving average (EMA) smoothing to ensure stability over time.

The results show that the system maintains consistently high performance across different chunks. The F1-score typically ranges between approximately 0.91 and 0.95, demonstrating stable detection capability under varying traffic conditions.

The dynamic threshold adapts continuously to changes in traffic behavior. In periods of higher anomaly activity, the threshold increases to reduce false positives, while in normal conditions, it decreases to maintain sensitivity to potential attacks.

In addition, the system achieves very low false positive rates across chunks, often approaching zero in some cases. This indicates that the adaptive threshold mechanism effectively controls false alarms without significantly compromising detection performance.

Overall, the real-time evaluation demonstrates that the proposed system is capable of maintaining high detection accuracy, stability, and adaptability in dynamic IoT environments. It should be noted that the real-time evaluation is conducted in a simulated environment using chunk-based processing. While this setup approximates real-world conditions, further validation in live deployment scenarios is required.

The observed variation in performance metrics across chunks reflects changes in traffic distribution and attack intensity. Despite these variations, the system maintains robust performance, indicating its ability to generalize well and adapt to different operational conditions.

Table 8 : Real-Time Performance Across Chunks

Metric	Range
F1-score	0.91 – 0.95
Accuracy	0.99 – 0.995
FPR	~0.000 – 0.005
ROC-AUC	~0.96 – 0.99

Table 8 summarizes the variation of performance metrics across different data chunks.

5.4 Discussion (Strengths and Limitations)

This section provides a detailed discussion of the strengths and limitations of the proposed intrusion detection system, based on the experimental results and real-time evaluation.

One of the main strengths of the system is its hybrid architecture, which combines supervised, unsupervised, and statistical detection techniques. This allows the system to capture different types of malicious behavior, including both known and unknown attacks. The integration of Random Forest, Isolation Forest, and KL Divergence improves overall detection accuracy and robustness.

Another key advantage is the use of a stacking model, which learns how to optimally combine the outputs of different models. This data-driven approach enhances decision-making and eliminates the need for manually tuning weights.

The dynamic threshold mechanism also plays a critical role in improving system performance. By adapting to changes in traffic behavior, the system maintains a balance between detection sensitivity and false positive reduction. This is particularly important in IoT environments, where traffic patterns are highly dynamic.

In addition, the system demonstrates strong real-time performance through chunk-based processing. This enables continuous monitoring of network traffic while maintaining computational efficiency and scalability.

Despite these strengths, the system has some limitations. First, the performance depends on the quality of the selected features. If important features are missing, detection performance may be affected.

Second, the system relies on labeled data for training the supervised component, which may limit its ability to generalize to entirely new types of attacks.

Third, the choice of chunk size may influence system behavior. While the selected chunk size provides a good balance, different environments may require further tuning to achieve optimal performance.

Finally, although the system performs well under the tested conditions, further validation on different datasets and real-world deployments is required to fully assess its generalizability.

Overall, the proposed system provides a robust and adaptive solution for intrusion detection in IoT environments, with strong performance and practical applicability.

The combination of adaptive thresholding and hybrid modeling represents a key contribution of this work, as it addresses both detection accuracy and system stability in dynamic environments. This makes the proposed approach more suitable for real-world IoT applications compared to traditional static detection methods.

Threats to validity include potential dataset bias, limited diversity of attack scenarios, and dependency on selected features. These factors may affect the generalization capability of the model to different environments.

Future work should focus on evaluating the system using multiple datasets and real-world deployments to further validate its robustness.

Chapter 6 — Conclusion and Future Work

6.1 Conclusion

This work presents a hybrid intrusion detection system designed for securing MQTT-based IoT environments. The system integrates multiple detection techniques, including Random Forest, Isolation Forest, and KL Divergence, combined through a stacking model to enhance detection performance.

The proposed system demonstrates strong performance in detecting both known and unknown attacks, while maintaining a low false positive rate. The use of a dynamic threshold further improves adaptability to changing network conditions, making the system suitable for real-time applications.

The main contributions of this work can be summarized as follows:

- Development of a hybrid intrusion detection model combining supervised, unsupervised, and statistical approaches.
- Implementation of a stacking-based integration method that learns how to optimally combine model outputs.
- Introduction of a dynamic threshold mechanism that adapts to traffic behavior using percentile-based analysis and smoothing.
- Design of a real-time detection system using chunk-based processing for efficient handling of streaming data.
- Achievement of high detection performance, with an F1-score of approximately 94% and a low false positive rate.

Overall, the proposed system provides a robust, adaptive, and efficient solution for intrusion detection in IoT networks.

6.2 Future Work

Despite the strong performance of the proposed system, several improvements can be considered for future work to further enhance its capabilities:

- Applying deep learning techniques, such as LSTM or autoencoders, to capture temporal patterns in network traffic.
- Evaluating the system on additional real-world datasets to improve generalization and robustness.
- Optimizing the dynamic threshold mechanism using advanced adaptive or reinforcement learning techniques.
- Deploying the system in a real IoT environment to assess its performance under live network conditions.
- Enhancing feature selection methods using automated or embedded approaches to improve model efficiency.
- Reducing computational overhead to support deployment on resource-constrained IoT devices.
- Integrating the system with automated response mechanisms to enable real-time mitigation of detected attacks.

These improvements can further strengthen the system and extend its applicability to a wider range of IoT security scenarios.

References

- [1] A. Alshamrani, M. Myneni, A. Chowdhary, and D. Huang, "A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and Research Opportunities," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2023.
- [2] S. Sicari, A. Rizzardi, L. Grieco, and A. Coen-Porisini, "Security, Privacy and Trust in Internet of Things: The Road Ahead," *IEEE Internet of Things Journal*, vol. 10, no. 1, pp. 1–18, 2024.
- [3] M. Conti, A. Dehghantanha, K. Franke, and S. Watson, "Internet of Things Security and Forensics: Challenges and Opportunities," *Future Generation Computer Systems*, vol. 140, pp. 123–135, 2023.
- [4] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [5] M. Ammar, G. Russello, and B. Crispo, "Internet of Things: A Survey on the Security of IoT Frameworks," *Journal of Information Security and Applications*, vol. 72, 2023.
- [6] A. Banks and R. Gupta, "MQTT Version 3.1.1 Protocol Specification," *OASIS Standard*, 2014.
- [7] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao, "A Survey on Internet of Things: Architecture, Enabling Technologies, Security and Privacy, and Applications," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 3050–3075, 2024.
- [8] H. Boyes, B. Hallaq, J. Cunningham, and T. Watson, "The Industrial Internet of Things (IIoT): An Analysis Framework," *Computers in Industry*, vol. 140, 2023.
- [9] R. Mitchell and I. Chen, "A Survey of Intrusion Detection Techniques for Cyber-Physical Systems," *ACM Computing Surveys*, vol. 55, no. 2, 2023.
- [10] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges," *Cybersecurity Journal*, vol. 7, no. 1, 2024.

- [11] Y. Meidan et al., "N-BaIoT: Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders," *IEEE Internet of Things Journal*, 2023.
- [12] S. Dua and X. Du, "Data Mining and Machine Learning in Cybersecurity," CRC Press, 2023.
- [13] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation Forest," in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.
- [14] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ, USA: Wiley, 2006..
- [15] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, 2023.
- [16] H. Hindy et al., "A Taxonomy of Network Threats and Intrusion Detection Systems," *IEEE Access*, vol. 11, 2023.
- [17] A. Ferrag, L. Maglaras, and H. Janicke, "Deep Learning for Cyber Security Intrusion Detection," *Journal of Information Security and Applications*, vol. 63, 2024.
- [18] S. García, M. Grill, J. Stiborek, and A. Zunino, "An Empirical Comparison of Botnet Detection Methods," *Computers & Security*, 2023.
- [19] J. Gama et al., "A Survey on Concept Drift Adaptation," *ACM Computing Surveys*, vol. 46, no. 4, 2023.
- [20] S. Verma, K. Dutta, and A. Sharma, "Lightweight Machine Learning-Based Intrusion Detection for IoT Systems," *Future Generation Computer Systems*, vol. 150, pp. 100–110, 2025.
- [21] H. Zhou, Y. Chen, and L. Wang, "A Hybrid Intrusion Detection Model Based on Machine Learning for IoT Networks," *IEEE Access*, vol. 12, pp. 45000–45012, 2024.
- [22] K. Singh, R. Patel, and M. Shah, "Adaptive Intrusion Detection System Using Dynamic Thresholding in IoT," *Computers & Security*, vol. 140, 2025.
- [23] I. Vaccari, V. Vasileios, and A. Ioannis, "Anomaly Detection in MQTT-Based IoT Systems," *IEEE Access*, 2023.